

## Sequential Circuits

```
-- =====
-- SEQUENTIAL CIRCUITS PRACTICE PROBLEMS
-- VHDL IMPLEMENTATIONS
-- =====

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- =====
-- PROBLEM 1: CUSTOM SEQUENCE COUNTER
-- Sequence: 0, 2, 5, 6, 0, 2, 5, 6, ...
-- Using JK Flip-Flops
-- =====

-- JK Flip-Flop Component
entity jk_flipflop is
    Port (
        J : in STD_LOGIC;
        K : in STD_LOGIC;
        CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;
        Q : out STD_LOGIC;
        Q_BAR : out STD_LOGIC
    );
end jk_flipflop;

architecture Behavioral of jk_flipflop is
    signal Q_internal : STD_LOGIC := '0';
begin
    process(CLK, RESET)
    begin
        if RESET = '1' then
            Q_internal <= '0';
        elsif rising_edge(CLK) then
            if J = '0' and K = '0' then
                Q_internal <= Q_internal; -- No change
            elsif J = '0' and K = '1' then
                Q_internal <= '0'; -- Reset
            elsif J = '1' and K = '0' then
                Q_internal <= '1'; -- Set
            else -- J = '1' and K = '1'

```

```

        Q_internal <= not Q_internal;  -- Toggle
    end if;
end if;
end process;

Q <= Q_internal;
Q_BAR <= not Q_internal;
end Behavioral;

-- Custom Sequence Counter (0, 2, 5, 6)
entity custom_sequence_counter is
    Port (
        CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;
        COUNT : out STD_LOGIC_VECTOR(2 downto 0)
    );
end custom_sequence_counter;

architecture Behavioral of custom_sequence_counter is
    component jk_flipflop is
        Port (
            J : in STD_LOGIC;
            K : in STD_LOGIC;
            CLK : in STD_LOGIC;
            RESET : in STD_LOGIC;
            Q : out STD_LOGIC;
            Q_BAR : out STD_LOGIC
        );
    end component;

    signal Q2, Q1, Q0 : STD_LOGIC;
    signal Q2_BAR, Q1_BAR, Q0_BAR : STD_LOGIC;
    signal J2, K2, J1, K1, J0, K0 : STD_LOGIC;

begin
    -- State assignment: 0=000, 2=010, 5=101, 6=110
    -- Next state logic derived from state table:

    -- J0 = Q2'Q1 + Q2Q1'      K0 = Q2'Q1 + Q2Q1'
    J0 <= (Q2_BAR and Q1) or (Q2 and Q1_BAR);
    K0 <= (Q2_BAR and Q1) or (Q2 and Q1_BAR);

    -- J1 = Q2'Q0'           K1 = Q2Q0'
    J1 <= Q2_BAR and Q0_BAR;
    K1 <= Q2 and Q0_BAR;

```

```

-- J2 = Q1Q0'          K2 = Q1'Q0
J2 <= Q1 and Q0_BAR;
K2 <= Q1_BAR and Q0;

-- Instantiate JK Flip-Flops
FF2: jk_flipflop port map (J2, K2, CLK, RESET, Q2, Q2_BAR);
FF1: jk_flipflop port map (J1, K1, CLK, RESET, Q1, Q1_BAR);
FF0: jk_flipflop port map (J0, K0, CLK, RESET, Q0, Q0_BAR);

COUNT <= Q2 & Q1 & Q0;
end Behavioral;

-- =====
-- PROBLEM 2: SEQUENCE DETECTOR "1011"
-- Moore Machine Implementation
-- =====

entity sequence_detector_1011 is
    Port (
        CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;
        DATA_IN : in STD_LOGIC;
        DETECTED : out STD_LOGIC
    );
end sequence_detector_1011;

architecture Behavioral of sequence_detector_1011 is
    type state_type is (S0, S1, S2, S3, S4);
    signal current_state, next_state : state_type;

begin
    -- State register
    state_register: process(CLK, RESET)
    begin
        if RESET = '1' then
            current_state <= S0;
        elsif rising_edge(CLK) then
            current_state <= next_state;
        end if;
    end process;

    -- Next state logic
    next_state_logic: process(current_state, DATA_IN)
    begin
        case current_state is
            when S0 => -- Initial state

```

```

        if DATA_IN = '1' then
            next_state <= S1; -- Got first '1'
        else
            next_state <= S0; -- Stay in S0
        end if;

    when S1 => -- Received "1"
        if DATA_IN = '0' then
            next_state <= S2; -- Got "10"
        else
            next_state <= S1; -- Stay in S1 (still have '1')
        end if;

    when S2 => -- Received "10"
        if DATA_IN = '1' then
            next_state <= S3; -- Got "101"
        else
            next_state <= S0; -- Back to start
        end if;

    when S3 => -- Received "101"
        if DATA_IN = '1' then
            next_state <= S4; -- Got "1011" - DETECTED!
        else
            next_state <= S2; -- Got "1010", go to S2
        end if;

    when S4 => -- Sequence detected
        if DATA_IN = '1' then
            next_state <= S1; -- New sequence might start
        else
            next_state <= S0; -- Back to start
        end if;

    when others =>
        next_state <= S0;
    end case;
end process;

-- Output logic (Moore machine - output depends only on state)
output_logic: process(current_state)
begin
    case current_state is
        when S4 =>
            DETECTED <= '1'; -- Sequence detected
        when others =>

```

```

        DETECTED <= '0'; -- No detection
    end case;
end process;

end Behavioral;

-- =====
-- PROBLEM 3: TRAFFIC LIGHT CONTROLLER
-- 4-Way Intersection Controller
-- =====

entity traffic_light_controller is
    Port (
        CLK : in STD_LOGIC;           -- System clock (assume 1 Hz for timing)
        RESET : in STD_LOGIC;
        EMERGENCY : in STD_LOGIC;      -- Emergency override
        PED_REQUEST : in STD_LOGIC;    -- Pedestrian crossing request
        NS_RED : out STD_LOGIC;        -- North-South Red
        NS_YELLOW : out STD_LOGIC;     -- North-South Yellow
        NS_GREEN : out STD_LOGIC;      -- North-South Green
        EW_RED : out STD_LOGIC;        -- East-West Red
        EW_YELLOW : out STD_LOGIC;     -- East-West Yellow
        EW_GREEN : out STD_LOGIC;      -- East-West Green
        WALK : out STD_LOGIC           -- Pedestrian walk signal
    );
end traffic_light_controller;

architecture Behavioral of traffic_light_controller is
    type state_type is (NS_GREEN_ST, NS_YELLOW_ST, EW_GREEN_ST, EW_YELLOW_ST, PED_WALK_ST);
    signal current_state, next_state : state_type;
    signal timer : integer range 0 to 35 := 0;
    signal ped_req_reg : STD_LOGIC := '0';

    -- Timing constants (in seconds)
    constant NS_GREEN_TIME : integer := 30;
    constant NS_YELLOW_TIME : integer := 5;
    constant EW_GREEN_TIME : integer := 25;
    constant EW_YELLOW_TIME : integer := 5;
    constant PED_WALK_TIME : integer := 15;

begin
    -- State register and timer
    state_timer_process: process(CLK, RESET)
    begin
        if RESET = '1' then
            current_state <= NS_GREEN_ST;

```

```

    timer <= 0;
    ped_req_reg <= '0';
elsif rising_edge(CLK) then
    if EMERGENCY = '1' then
        current_state <= NS_GREEN_ST;  -- Emergency: default to NS green
        timer <= 0;
    else
        current_state <= next_state;
        if current_state /= next_state then
            timer <= 0;  -- Reset timer on state change
        else
            timer <= timer + 1;
        end if;

        -- Latch pedestrian request
        if PED_REQUEST = '1' then
            ped_req_reg <= '1';
        elsif current_state = PED_WALK_ST then
            ped_req_reg <= '0';  -- Clear after pedestrian phase
        end if;
    end if;
end if;
end process;

-- Next state logic
next_state_logic: process(current_state, timer, ped_req_reg)
begin
    case current_state is
        when NS_GREEN_ST =>
            if timer >= NS_GREEN_TIME - 1 then
                if ped_req_reg = '1' then
                    next_state <= PED_WALK_ST;
                else
                    next_state <= NS_YELLOW_ST;
                end if;
            else
                next_state <= NS_GREEN_ST;
            end if;
        when NS_YELLOW_ST =>
            if timer >= NS_YELLOW_TIME - 1 then
                next_state <= EW_GREEN_ST;
            else
                next_state <= NS_YELLOW_ST;
            end if;
    end case;
end process;

```

```

when EW_GREEN_ST =>
    if timer >= EW_GREEN_TIME - 1 then
        next_state <= EW_YELLOW_ST;
    else
        next_state <= EW_GREEN_ST;
    end if;

when EW_YELLOW_ST =>
    if timer >= EW_YELLOW_TIME - 1 then
        next_state <= NS_GREEN_ST;
    else
        next_state <= EW_YELLOW_ST;
    end if;

when PED_WALK_ST =>
    if timer >= PED_WALK_TIME - 1 then
        next_state <= NS_YELLOW_ST;
    else
        next_state <= PED_WALK_ST;
    end if;

when others =>
    next_state <= NS_GREEN_ST;
end case;
end process;

-- Output logic
output_logic: process(current_state, EMERGENCY)
begin
    if EMERGENCY = '1' then
        -- Emergency mode: All red except NS green
        NS_RED <= '0';
        NS_YELLOW <= '0';
        NS_GREEN <= '1';
        EW_RED <= '1';
        EW_YELLOW <= '0';
        EW_GREEN <= '0';
        WALK <= '0';
    else
        case current_state is
            when NS_GREEN_ST =>
                NS_RED <= '0';
                NS_YELLOW <= '0';
                NS_GREEN <= '1';
                EW_RED <= '1';
                EW_YELLOW <= '0';
        end case;
    end if;
end process;

```

```

EW_GREEN <= '0';
WALK <= '0';

when NS_YELLOW_ST =>
    NS_RED <= '0';
    NS_YELLOW <= '1';
    NS_GREEN <= '0';
    EW_RED <= '1';
    EW_YELLOW <= '0';
    EW_GREEN <= '0';
    WALK <= '0';

when EW_GREEN_ST =>
    NS_RED <= '1';
    NS_YELLOW <= '0';
    NS_GREEN <= '0';
    EW_RED <= '0';
    EW_YELLOW <= '0';
    EW_GREEN <= '1';
    WALK <= '0';

when EW_YELLOW_ST =>
    NS_RED <= '1';
    NS_YELLOW <= '0';
    NS_GREEN <= '0';
    EW_RED <= '0';
    EW_YELLOW <= '1';
    EW_GREEN <= '0';
    WALK <= '0';

when PED_WALK_ST =>
    NS_RED <= '1';
    NS_YELLOW <= '0';
    NS_GREEN <= '0';
    EW_RED <= '1';
    EW_YELLOW <= '0';
    EW_GREEN <= '0';
    WALK <= '1';

when others =>
    NS_RED <= '1';
    NS_YELLOW <= '0';
    NS_GREEN <= '0';
    EW_RED <= '1';
    EW_YELLOW <= '0';
    EW_GREEN <= '0';

```

```

        WALK <= '0';
    end case;
end if;
end process;

end Behavioral;

-- =====
-- PROBLEM 4: UART TRANSMITTER
-- 8-bit data, 1 start bit, 1 stop bit
-- =====

entity uart_transmitter is
    Port (
        CLK : in STD_LOGIC;           -- System clock
        RESET : in STD_LOGIC;
        BAUD_CLK : in STD_LOGIC;      -- Baud rate clock
        DATA_IN : in STD_LOGIC_VECTOR(7 downto 0);
        START_TX : in STD_LOGIC;      -- Start transmission
        TX_DATA : out STD_LOGIC;      -- Serial output
        TX_BUSY : out STD_LOGIC;      -- Transmission in progress
        TX_DONE : out STD_LOGIC;      -- Transmission complete
    );
end uart_transmitter;

architecture Behavioral of uart_transmitter is
    type state_type is (IDLE, START_BIT, DATA_BITS, STOP_BIT);
    signal current_state, next_state : state_type;
    signal shift_reg : STD_LOGIC_VECTOR(7 downto 0);
    signal bit_counter : integer range 0 to 7;
    signal data_reg : STD_LOGIC_VECTOR(7 downto 0);
    signal tx_start_sync : STD_LOGIC_VECTOR(1 downto 0);
    signal start_detected : STD_LOGIC;

begin
    -- Synchronize start signal to baud clock domain
    sync_start: process(BAUD_CLK, RESET)
    begin
        if RESET = '1' then
            tx_start_sync <= "00";
        elsif rising_edge(BAUD_CLK) then
            tx_start_sync <= tx_start_sync(0) & START_TX;
        end if;
    end process;

    start_detected <= tx_start_sync(0) and not tx_start_sync(1);

```

```

-- Latch input data
data_latch: process(CLK, RESET)
begin
    if RESET = '1' then
        data_reg <= (others => '0');
    elsif rising_edge(CLK) then
        if START_TX = '1' and current_state = IDLE then
            data_reg <= DATA_IN;
        end if;
    end if;
end process;

-- State machine
state_machine: process(BAUD_CLK, RESET)
begin
    if RESET = '1' then
        current_state <= IDLE;
        shift_reg <= (others => '1');
        bit_counter <= 0;
    elsif rising_edge(BAUD_CLK) then
        case current_state is
            when IDLE =>
                if start_detected = '1' then
                    current_state <= START_BIT;
                    shift_reg <= data_reg;
                    bit_counter <= 0;
                end if;

            when START_BIT =>
                current_state <= DATA_BITS;
                bit_counter <= 0;

            when DATA_BITS =>
                shift_reg <= '1' & shift_reg(7 downto 1); -- Right shift
                if bit_counter = 7 then
                    current_state <= STOP_BIT;
                    bit_counter <= 0;
                else
                    bit_counter <= bit_counter + 1;
                end if;

            when STOP_BIT =>
                current_state <= IDLE;

            when others =>

```

```

        current_state <= IDLE;
    end case;
end if;
end process;

-- Output logic
output_logic: process(current_state, shift_reg)
begin
    case current_state is
        when IDLE =>
            TX_DATA <= '1';           -- Idle high
            TX_BUSY <= '0';
            TX_DONE <= '0';

        when START_BIT =>
            TX_DATA <= '0';           -- Start bit (low)
            TX_BUSY <= '1';
            TX_DONE <= '0';

        when DATA_BITS =>
            TX_DATA <= shift_reg(0); -- LSB first
            TX_BUSY <= '1';
            TX_DONE <= '0';

        when STOP_BIT =>
            TX_DATA <= '1';           -- Stop bit (high)
            TX_BUSY <= '1';
            TX_DONE <= '1';

        when others =>
            TX_DATA <= '1';
            TX_BUSY <= '0';
            TX_DONE <= '0';
    end case;
end process;

end Behavioral;

-- =====
-- BAUD RATE GENERATOR
-- Generates baud rate clock from system clock
-- =====

entity baud_rate_generator is
    generic (
        CLK_FREQ : integer := 50_000_000; -- System clock frequency (Hz)

```

```

        BAUD_RATE : integer := 115200           -- Desired baud rate
    );
    Port (
        CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;
        BAUD_CLK : out STD_LOGIC
    );
end baud_rate_generator;

architecture Behavioral of baud_rate_generator is
    constant DIVISOR : integer := CLK_FREQ / BAUD_RATE;
    signal counter : integer range 0 to DIVISOR - 1;
    signal baud_clk_internal : STD_LOGIC := '0';

begin
    process(CLK, RESET)
    begin
        if RESET = '1' then
            counter <= 0;
            baud_clk_internal <= '0';
        elsif rising_edge(CLK) then
            if counter = DIVISOR - 1 then
                counter <= 0;
                baud_clk_internal <= not baud_clk_internal;
            else
                counter <= counter + 1;
            end if;
        end if;
    end process;

    BAUD_CLK <= baud_clk_internal;
end Behavioral;

-- =====
-- COMPLETE UART SYSTEM
-- Combines transmitter with baud rate generator
-- =====

entity uart_system is
    generic (
        CLK_FREQ : integer := 50_000_000; -- 50 MHz system clock
        BAUD_RATE : integer := 115200      -- 115200 baud
    );
    Port (
        CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;

```

```

        DATA_IN : in STD_LOGIC_VECTOR(7 downto 0);
        START_TX : in STD_LOGIC;
        TX_DATA  : out STD_LOGIC;
        TX_BUSY  : out STD_LOGIC;
        TX_DONE  : out STD_LOGIC
    );
end uart_system;

architecture Structural of uart_system is
    component baud_rate_generator is
        generic (
            CLK_FREQ : integer := 50_000_000;
            BAUD_RATE : integer := 115200
        );
        Port (
            CLK : in STD_LOGIC;
            RESET : in STD_LOGIC;
            BAUD_CLK : out STD_LOGIC
        );
    end component;

    component uart_transmitter is
        Port (
            CLK : in STD_LOGIC;
            RESET : in STD_LOGIC;
            BAUD_CLK : in STD_LOGIC;
            DATA_IN : in STD_LOGIC_VECTOR(7 downto 0);
            START_TX : in STD_LOGIC;
            TX_DATA : out STD_LOGIC;
            TX_BUSY : out STD_LOGIC;
            TX_DONE : out STD_LOGIC
        );
    end component;

    signal baud_clk : STD_LOGIC;

begin
    -- Instantiate baud rate generator
    BAUD_GEN: baud_rate_generator
        generic map (
            CLK_FREQ => CLK_FREQ,
            BAUD_RATE => BAUD_RATE
        )
        port map (
            CLK => CLK,
            RESET => RESET,

```

```

        BAUD_CLK => baud_clk
    );

    -- Instantiate UART transmitter
    UART_TX: uart_transmitter
    port map (
        CLK => CLK,
        RESET => RESET,
        BAUD_CLK => baud_clk,
        DATA_IN => DATA_IN,
        START_TX => START_TX,
        TX_DATA => TX_DATA,
        TX_BUSY => TX_BUSY,
        TX_DONE => TX_DONE
    );

end Structural;

```